

Draft - Settlement API

Created By	@Mukesh Kumar Yadav	
Reviewed By	@Sebastiano Serri	

Settlement Service API Design

- Settlement Service API Design
 - 1. Overview
 - 2. Recommended API coverage
 - 3. End-to-end journey
 - 4. Sequence diagram
 - 5. API details
 - 5.1 Create allocation settlement
 - 5.2 Get Settlement
 - 5.3 Update Custom Amount Settlement
 - 5.4 Cancel Settlement
 - 5.5 List Settlement Allocations
 - 5.6 Apply Credit Note
 - 5.7 Remove Credit Note
 - 5.8 Submit Settlement
 - 5.9 Settlement History
 - 6. Validation and business rules
 - 7. Database Changes
 - Option 1 — Pay selected invoices
 - Option 2 — Pay custom amount
 - 8. Existing Invoice Calculation
 - 9. Settlement Database Tables
 - 10. Settlement Table
 - 11. Settlement Invoice Allocation
 - 12. Settlement Credit Note Allocation
 - 13. Complete Database Relationship
 - 14. Step 1 – User Starts Payment
 - 15. Step 2 – Invoice Payment Path
 - 16. Step 3 – User Selects Invoices
 - 17. Step 4 – Calculate Invoice Amount
 - 18. Step 5 – Create Settlement
 - 19. Step 6 – Save Invoice Allocations
 - 20. Step 7 – Display Credit Notes
 - 21. Step 8 – User Selects Credit Notes
 - 22. Step 9 – Calculate Credit Note Total

- [23. Step 10 – Calculate Payable Amount](#)
- [24. Step 12 – Create Payment](#)
- [25. Step 14 – Successful Payment](#)
- [26. Step 15 – Failed Payment](#)
- [27. Custom Amount Flow](#)
- [28. Custom Settlement](#)
- [29. Custom Payment](#)
- [30. Custom Payment Flow Diagram](#)
- [31. Settlement Status](#)
- [32. Payment Status](#)
- [33. Complete Invoice Flow](#)
- [34. Complete Database Flow](#)
- [Swagger OPENAPI Specs:](#)

1. Overview

The Settlement API manages the settlement lifecycle for trade-credit accounts. A settlement specifies what a customer will pay: selected invoices, the total amount due, or a custom amount. Credit notes may reduce the final payable amount.

This API creates draft settlements, returns settlement details, updates custom-amount settlements, cancels draft settlements, and applies or removes credit notes.

2. Recommended API coverage

Area	Recommended API	Description
Create settlement	PUT <code>/v1/accounts/{accountId}/settlements</code>	Creates a new settlement in DRAFT status based on selected invoices, full due amount, or a custom amount.
Get settlement	GET <code>/v1/accounts/{accountId}/settlements/{settlementId}</code>	Returns the settlement details including the invoices and credit notes allocation, totals and status.
Update a custom amount settlement	PATCH <code>/v1/accounts/{accountId}/settlements/{settlementId}</code>	Updates the amount and/or customer reference for a CUSTOM_AMOUNT settlement while it remains in DRAFT status.

Cancels a draft settlement	DELETE /v1/accounts/{accountId}/settlements/{settlementId}	Cancels a draft settlement and sets its status to DELETED.
Return the paged list of selected invoice or credit notes	GET /v1/accounts/{accountId}/settlements/{settlementId}/allocations	Returns a paginated list of the settlement's allocated invoices or credit-notes.
Apply a credit note to a settlement	PUT /v1/accounts/{accountId}/settlements/{settlementId}/credit-notes/{creditNoteId}	Allocates a credit note to the settlement and recalculates the outstanding payable amount.
Remove a credit note from a settlement	DELETE /v1/accounts/{accountId}/settlements/{settlementId}/credit-notes/{creditNoteId}	Removes an allocated credit note from the settlement and recalculates the settlement totals.
Submit a settlement for payment	POST /v1/accounts/{accountId}/settlements/{settlementId}/submit	Validates and submits the settlement for payment processing, creating an associated payment transaction.
Settlements history	GET /v1/accounts/{accountId}/settlements	Returns the paginated list of the settlements associated to the account.

3. End-to-end journey

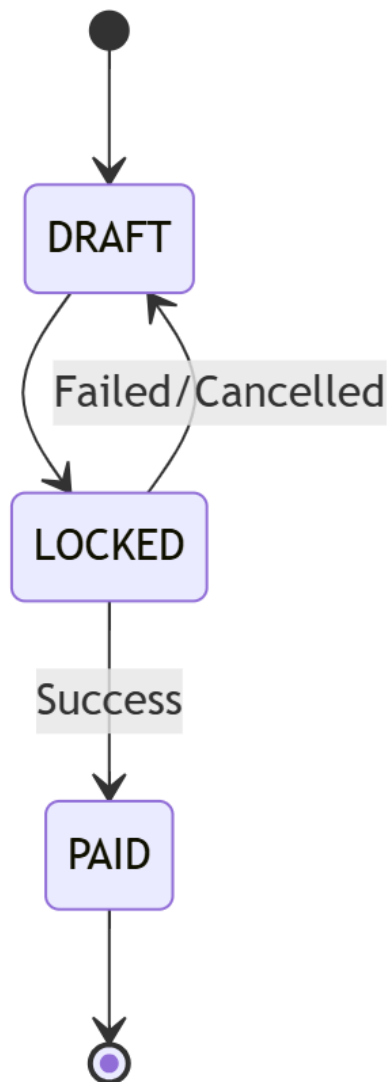
Settlement Lifecycle

The API defines the following settlement states:

Status	Description
DRAFT	Settlement is currently edited.

LOCKED	Payment has been initiated and settlement editing is blocked.
SETTLED	Payment completed successfully.

```
1 stateDiagram-v2
2   [*] --> DRAFT
3   DRAFT --> LOCKED
4   LOCKED --> PAID : Success
5   LOCKED --> DRAFT : Failed/Cancelled
6   PAID --> [*]
```



```
graph TD
    Customer --> CreateDRAFT[Create DRAFT settlement]
    CreateDRAFT --> Browse[Browse invoices + credit notes with settlementId]
```

```

8      |
9      v
10     Add/remove allocations
11     |
12     v
13     Settlement API recalculates debit, credit and payable totals
14     |
15     v
16     Choose payment method and saved card if required
17     |
18     v
19     POST /pay returns paymentId + hostedPageUrl
20     |
21     v
22     Customer completes hosted payment
23     |
24     v
25     App polls payment status
26     |
27     v
28     Refresh settlement + selected allocations
29     |
30     v
31     Show success / pending / failure screen

```

4. Sequence diagram

```

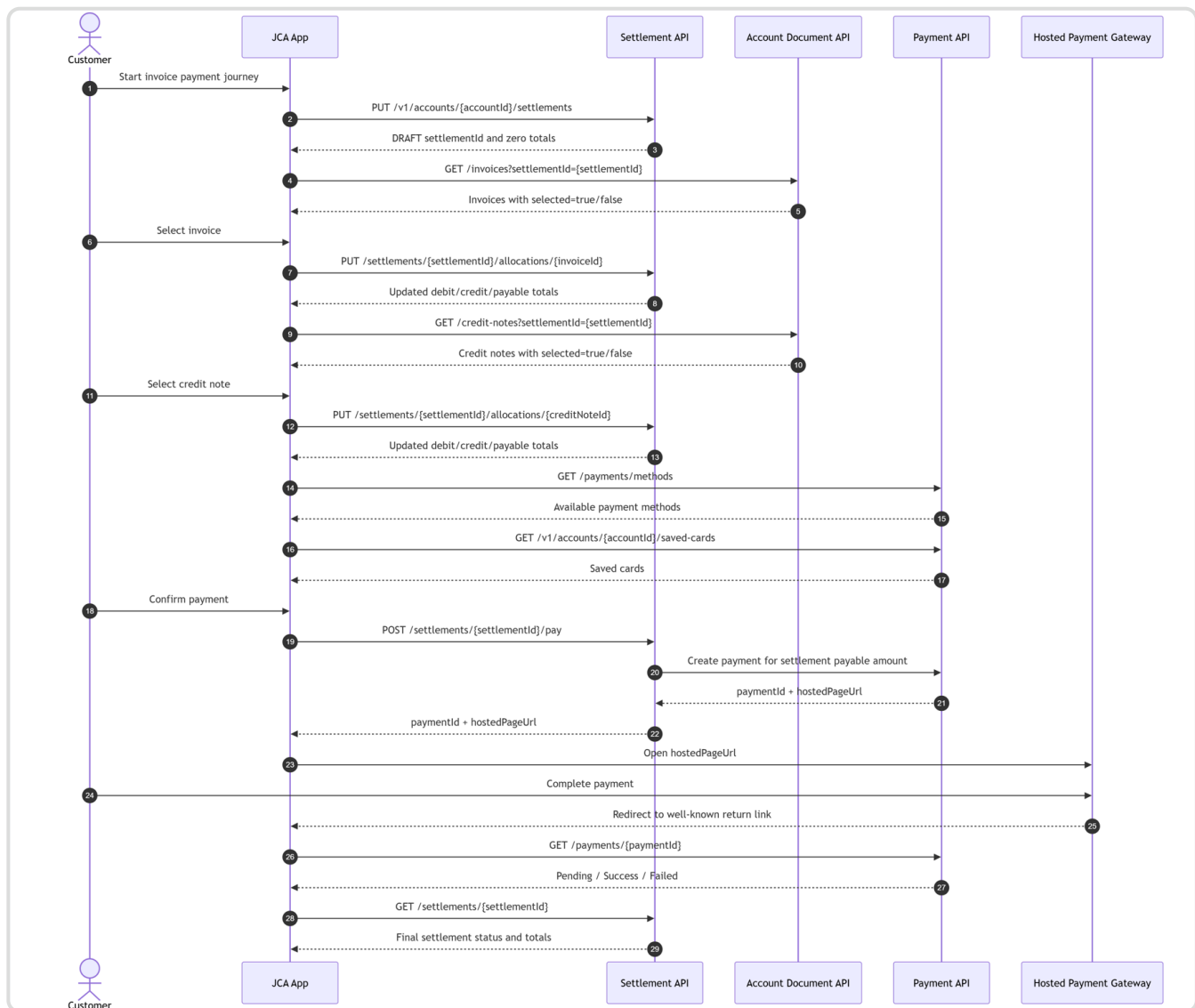
1  sequenceDiagram
2      autonumber
3      actor Customer
4      participant App as JCA App
5      participant SettlementAPI as Settlement API
6      participant DocumentAPI as Account Document API
7      participant PaymentAPI as Payment API
8      participant Gateway as Hosted Payment Gateway
9
10     Customer->>App: Start invoice payment journey
11     App->>SettlementAPI: PUT /v1/accounts/{accountId}/settlements
12     SettlementAPI-->>App: DRAFT settlementId and zero totals
13
14     App->>DocumentAPI: GET /invoices?settlementId={settlementId}
15     DocumentAPI-->>App: Invoices with selected=true/false
16     Customer->>App: Select invoice
17     App->>SettlementAPI: PUT /settlements/{settlementId}/allocations/{invoiceId}
18     SettlementAPI-->>App: Updated debit/credit/payable totals
19
20     App->>DocumentAPI: GET /credit-notes?settlementId={settlementId}
21     DocumentAPI-->>App: Credit notes with selected=true/false
22     Customer->>App: Select credit note
23     App->>SettlementAPI: PUT /settlements/{settlementId}/allocations/{creditNoteId}
24     SettlementAPI-->>App: Updated debit/credit/payable totals
25
26     App->>PaymentAPI: GET /payments/methods
27     PaymentAPI-->>App: Available payment methods
28     App->>PaymentAPI: GET /v1/accounts/{accountId}/saved-cards
29     PaymentAPI-->>App: Saved cards
30
31     Customer->>App: Confirm payment
32     App->>SettlementAPI: POST /settlements/{settlementId}/pay
33     SettlementAPI->>PaymentAPI: Create payment for settlement payable amount
34     PaymentAPI-->>SettlementAPI: paymentId + hostedPageUrl
35     SettlementAPI-->>App: paymentId + hostedPageUrl
36
37     App->>Gateway: Open hostedPageUrl
38     Customer->>Gateway: Complete payment
39     Gateway-->>App: Redirect to well-known return link

```

```

40 App->>PaymentAPI: GET /payments/{paymentId}
41 PaymentAPI-->>App: Pending / Success / Failed
42 App->>SettlementAPI: GET /settlements/{settlementId}
43 SettlementAPI-->>App: Final settlement status and totals

```



5. API details

5.1 Create allocation settlement

Endpoint: PUT /v1/accounts/{accountId}/settlements

Supported Types

	Type	Description
1	ALLOCATION	Customer selects specific invoices
2	DUE_AMOUNT	Pay entire outstanding balance
3	CUSTOM_AMOUNT	Pay custom amount

Request body:

Invoice Allocation Settlement:

```
1 {
2   "type": "ALLOCATION",
3   "invoiceIds": [
4     "00000000-0000-0000-0000-000000000001",
5     "00000000-0000-0000-0000-000000000002"
6   ]
7 }
```

Due Amount Settlement

```
1 {
2   "type": "DUE_AMOUNT"
3 }
```

Custom Amount Settlement

```
1 {
2   "type": "CUSTOM_AMOUNT",
3   "amount": 1000.00,
4   "customerReference": "P012345"
5 }
```

Response:

```
1 {
2   "id": "settlement-id",
3   "accountId": "account-id",
4   "type": "ALLOCATION",
5   "status": "DRAFT",
6   "invoicesTotalAmount": 3600.00,
7   "creditNotesTotalAmount": 0,
8   "totalAmount": 3600.00,
9   "expiresAt": "2026-05-19T14:30:00Z",
10  "createdAt": "2026-05-19T13:30:00Z"
11 }
12
```

Rules: Only one active draft allocation settlement should exist per account/session.

`accountId` must be validated against the logged-in session.

5.2 Get Settlement

Endpoint: GET `/v1/accounts/{accountId}/settlements/{settlementId}`

Query parameters: `accountId`, `settlementId`, `pageNumber`, `pageSize`

Description: Returns settlement details including invoices, credit notes, totals and current status.

```
1 {
2   "id": "settlement-id",
3   "accountId": "account-id",
4   "type": "ALLOCATION",
5   "status": "DRAFT",
6   "invoicesTotalAmount": 3600.00,
```

```

7   "creditNotesTotalAmount": 1200.00,
8   "totalAmount": 2400.00,
9   "invoiceAllocations": [],
10  "creditNotesAllocations": []
11 }

```

5.3 Update Custom Amount Settlement

Endpoint: PATCH /v1/accounts/{accountId}/settlements/{settlementId}

Description: Updates amount and customer reference for a CUSTOM_AMOUNT settlement.

Request body:

```

1  {
2    "amount": 999.00,
3    "customerReference": "Invoice#4005"
4  }

```

Response:

```

1  {
2    "id": "settlement-id",
3    "type": "CUSTOM_AMOUNT",
4    "status": "DRAFT",
5    "totalAmount": 999.00,
6    "customerReference": "Invoice#4005"
7  }

```

5.4 Cancel Settlement

Endpoint: DELETE /v1/accounts/{accountId}/settlements/{settlementId}

Description: Cancels a draft settlement.

Response:

```

1  204 No Content

```

5.5 List Settlement Allocations

Endpoint: GET

/v1/accounts/{accountId}/settlements/{settlementId}/allocations

Query Parameters

Parameter	Description
type	INVOICE / CREDIT_NOTE
pageNum	Page Number
pageSize	Page Size

Response:

```

1  {
2    "items": [
3      {
4        "invoiceId": "invoice-id",
5        "reference": "INV-0042",
6        "amount": 1200.00,
7        "dueDate": "2026-08-01"
8      }
9    ],
10   "pageNum": 1,
11   "pageSize": 10,
12   "hasMore": true
13 }

```

5.6 Apply Credit Note

Endpoint: PUT /v1/accounts/{accountId}/settlements/{settlementId}/credit-notes/{creditNoteId}

Description: Applies credit note against settlement and updates payable amount.

Query parameters: accountId, settlementId, creditNoteId

```

1  {
2    "id": "settlement-id",
3    "invoicesTotalAmount": 3600.00,
4    "creditNotesTotalAmount": 1200.00,
5    "totalAmount": 2400.00
6  }

```

5.7 Remove Credit Note

Endpoint: DELETE

/v1/accounts/{accountId}/settlements/{settlementId}/credit-notes/{creditNoteId}

Response:

```

1  {
2    "id": "settlement-id",
3    "creditNotesTotalAmount": 0,
4    "totalAmount": 3600.00
5  }

```

5.8 Submit Settlement

Endpoint: POST /v1/accounts/{accountId}/settlements/{settlementId}/submit

Description: Submits settlement for payment processing.

Request:

```

1  {
2    "paymentMethodId": "PM002",
3    "returnUrl": "app://payment-return"
4  }

```

5.9 Settlement History

Endpoint: GET /v1/accounts/{accountId}/settlements

Response:

```
1 {
2   "pageNum": 1,
3   "pageSize": 20,
4   "hasMore": true,
5   "items": [
6     {
7       "settlementId": "SET001",
8       "type": "ALLOCATION",
9       "status": "PAID",
10      "amount": 2400.00
11    },
12    {
13      "settlementId": "SET002",
14      "type": "CUSTOM_AMOUNT",
15      "status": "DRAFT",
16      "amount": 500.00
17    }
18  ]
19 }
```

6. Validation and business rules

- The settlement must belong to the current user/session for every settlement operation.
- The account in the path must match the account associated with the user session.
- Only one active draft settlement should exist per session/account to avoid conflicting selected states.
- Invoice and credit-note transactions must belong to the same account.
- Allocations are editable only while the settlement is in **DRAFT**.
- Payment cannot start if no invoice is selected, payable amount is invalid, or payment is already in progress.
- Credit-note total must not exceed invoice debit total unless the business explicitly supports account-credit residuals.
- Allocation add/remove should be idempotent where possible to make repeated mobile requests safe.
- Use a single status vocabulary across settlement and payment flows: either **PAYING** or **PAYMENT_PENDING**, not both unless clearly defined.

7. Database Changes

It captures the persistence model used to store settlement headers, invoice allocations, and credit-note allocations.

```
1 /***** Object: Table [dbo].[Settlements] *****/
2
3 SET ANSI_NULLS ON
4 GO
```

```

5
6 SET QUOTED_IDENTIFIER ON
7 GO
8
9 CREATE TABLE [dbo].[Settlements]
10 (
11     [id] [uniqueidentifier] NOT NULL CONSTRAINT [DF_Settlements_Id] DEFAULT (NEWID()),
12     [accountId] [varchar](50) NOT NULL,
13     [status] [nvarchar](30) NOT NULL,
14     [type] [nvarchar](30) NULL,
15     [amount] [decimal](18,2) NOT NULL,
16     [invoiceTotal] [decimal](18,2) NOT NULL,
17     [creditNoteTotal] [decimal](18,2) NOT NULL,
18     [creationDateTime] NOT NULL CONSTRAINT [DF_Settlements_CreationDateTime] DEFAULT
(SYSDATETIME()),
19     [updateDateTime] NOT NULL CONSTRAINT [DF_Settlements_UpdateDateTime] DEFAULT
(SYSDATETIME()),
20     CONSTRAINT [PK_Settlements]
21     PRIMARY KEY CLUSTERED
22     (
23         [id] ASC
24     )
25 ) ON [PRIMARY]
26 GO
27
28 /***** Object: Table [dbo].[SettlementsAllocation] *****/
29
30 SET ANSI_NULLS ON
31 GO
32
33 SET QUOTED_IDENTIFIER ON
34 GO
35
36 CREATE TABLE [dbo].[SettlementsAllocation]
37 (
38     [id] [uniqueidentifier] NOT NULL CONSTRAINT [DF_SettlementsAllocation_Id] DEFAULT
(NEWID()),
39     [settlementId] [uniqueidentifier] NOT NULL,
40     [invoiceId] [nvarchar](50) NOT NULL,
41     [creationDateTime] NOT NULL CONSTRAINT [DF_SettlementsAllocation_CreationDateTime]
DEFAULT (SYSDATETIME()),
42     [updateDateTime] NOT NULL CONSTRAINT [DF_SettlementsAllocation_UpdateDateTime]
DEFAULT SYSDATETIME(),
43     CONSTRAINT [PK_SettlementsAllocation]
44     PRIMARY KEY CLUSTERED
45     (
46         [id] ASC
47     ),
48     CONSTRAINT [FK_SettlementsAllocation_Settlement]
49     FOREIGN KEY ([settlementId])
50     REFERENCES [dbo].[Settlements] ([id])
51 ) ON [PRIMARY]
52 GO

```

Option 1 — Pay selected invoices

The user:

1. Searches available invoices.
2. Selects one or more invoices.
3. The system calculates the invoice amount.

4. The user can select available credit notes.
5. The system calculates the final payable amount.
6. The user selects a payment method.
7. The payment is processed through SmartPay.

Option 2 — Pay custom amount

The user:

1. Enters a custom amount.
2. Enters a payment reference.
3. Selects a payment method.
4. The payment is processed through SmartPay.

```
1 | dbo.Invoices_Headers
```

It contains both invoices and credit notes.

The `documentType` column determines the document:

```
1 | | documentType | Document |
2 | | ----- | ----- |
3 | | `S`         | Invoice   |
4 | | `C`         | Credit Note |
5 |
```

Therefore:

```
1 | Invoices_Headers
2 | |
3 | +--- documentType = S ---> Invoice
4 | |
5 | +--- documentType = C ---> Credit Note
```

There is **no separate CreditNotes table**.

8. Existing Invoice Calculation

The invoice amount is not stored directly as the settlement amount.

It is calculated using:

```
1 | Invoices_Headers
2 | |
3 | v
4 | Orders_Lines
5 | |
6 | v
7 | Orders_Line_Pricings
```

The existing calculation is based on:

```
1 | Original Quantity × List Price
```

plus VAT.

Conceptually:

```
1 Net Amount = SUM(OriginalQuantity × ListPrice)
2 VAT Amount = SUM(VatAmount)
3 Invoice Amount = Net Amount + VAT Amount
```

This calculation should be reused by the Settlement Service rather than creating another invoice calculation mechanism.

9. Settlement Database Tables

The Settlement feature requires three settlement-specific tables.

```
1 Settlement
2 |
3 +--- SettlementInvoiceAllocation
4 |
5 +--- SettlementCreditNoteAllocation
```

10. Settlement Table

The `Settlement` table represents the **overall payment/settlement transaction**.

Recommended structure:

Column	Type	Purpose
<code>`Id`</code>	<code>`UNIQUEIDENTIFIER`</code>	Settlement ID
<code>`AccountId`</code>	<code>`VARCHAR(50)`</code>	Customer account
<code>`Status`</code>	<code>`NVARCHAR(30)`</code>	Settlement status
<code>`PaymentType`</code>	<code>`NVARCHAR(20)`</code>	<code>`INVOICE`</code> / <code>`CUSTOM`</code>
<code>`CurrencyCode`</code>	<code>`CHAR(3)`</code>	Currency
<code>`InvoiceTotal`</code>	<code>`DECIMAL(18,2)`</code>	Total selected invoice amount
<code>`CreditNoteTotal`</code>	<code>`DECIMAL(18,2)`</code>	Total applied credit notes
<code>`CustomAmount`</code>	<code>`DECIMAL(18,2)`</code>	Custom payment amount
<code>`PayableAmount`</code>	<code>`DECIMAL(18,2)`</code>	Final amount to pay
<code>`CustomPaymentReference`</code>	<code>`NVARCHAR(100)`</code>	Custom payment reference
<code>`CreationDateTime`</code>	<code>`DATETIME2`</code>	Created date
<code>`UpdateDateTime`</code>	<code>`DATETIME2`</code>	Updated date

Important:

For invoice payments:

```
1 PaymentType = INVOICE
```

For custom payments:

```
1 PaymentType = CUSTOM
```

11. Settlement Invoice Allocation

This table represents:

Which invoices have been selected for this settlement?

1 SettlementInvoiceAllocation

Recommended structure:

Relationship:

```
1 Settlement
2 |
3 | 1
4 |
5 | many
6 v
7 SettlementInvoiceAllocation
8 |
9 | InvoiceId
10 v
11 Invoices_Headers
12 |
13 | documentType = S
14 v
15 Invoice
```

12. Settlement Credit Note Allocation

This table represents:

Which credit notes have been applied to this settlement?

1 SettlementsAllocation

Recommended structure:

Relationship:

```
1 Settlement
2 |
3 | 1
4 |
5 | many
6 v
7 SettlementsAllocation
8 |
9 | InvoiceId
10 v
11 Invoices_Headers
12 |
13 | documentType = C
14 v
15 Credit Note
```

13. Complete Database Relationship

Now the complete database relationship is simple:

1 erDiagram

```

3      Settlements {
4          UNIQUEIDENTIFIER id PK
5          VARCHAR accountId
6          NVARCHAR status
7          NVARCHAR type
8          DECIMAL amount
9          DECIMAL invoiceTotal
10         DECIMAL creditNoteTotal
11         DATETIME2 creationDateTime
12         DATETIME2 updationDateTime
13     }
14
15     SettlementsAllocation {
16         UNIQUEIDENTIFIER id PK
17         UNIQUEIDENTIFIER settlementId FK
18         NVARCHAR invoiceId
19         DATETIME2 creationDateTime
20         DATETIME2 updationDateTime
21     }
22
23     Invoices_Headers {
24         UNIQUEIDENTIFIER id PK
25         VARCHAR branchCode
26         VARCHAR customerAccount
27         VARCHAR accountType
28         NVARCHAR standardInvoiceNo
29         NVARCHAR consolidatedInvoiceNo
30         DATETIMEOFFSET invoiceDateTime
31         DATETIMEOFFSET dueDate
32         CHAR documentType
33         DATETIMEOFFSET createdOn
34     }
35
36     Orders_Lines {
37         UNIQUEIDENTIFIER id PK
38         UNIQUEIDENTIFIER invoiceHeaderId FK
39         UNIQUEIDENTIFIER orderHeaderId FK
40         INT lineNumber
41         NVARCHAR itemCode
42         DECIMAL originalQuantity
43         DECIMAL goodsValue
44     }
45
46     Orders_Line_Pricings {
47         UNIQUEIDENTIFIER id PK
48         UNIQUEIDENTIFIER orderLineId FK
49         DECIMAL listPrice
50         DECIMAL vatAmount
51         NVARCHAR vatCode
52         NVARCHAR unitForSale
53     }
54
55     Payments {
56         UNIQUEIDENTIFIER Id PK
57         UNIQUEIDENTIFIER CorrelationId
58         UNIQUEIDENTIFIER SystemPaymentMethodId FK
59         NVARCHAR DeviceId
60         NVARCHAR SessionId
61         UNIQUEIDENTIFIER UserId
62         UNIQUEIDENTIFIER SystemId
63         VARCHAR AccountId
64         NVARCHAR PaymentPurpose
65         UNIQUEIDENTIFIER PaymentPurposeReference
66         CHAR CurrencyCode
67         DECIMAL TotalAmount
68         NVARCHAR ResultPageUrl
69         NVARCHAR PaymentStatus

```

```
70         DATETIME CreationDatetime
71         DATETIME UpdationDatetime
72     }
73
74     Settlements ||--o{ SettlementsAllocation : contains
75     SettlementsAllocation }o..|| Invoices_Headers : "invoiceId / logical relationship"
76
77     Invoices_Headers ||--o{ Orders_Lines : contains
78
79     Orders_Lines ||--o| Orders_Line_Pricings : has_pricing
80
81     Settlements ||--o{ Payments : "PaymentPurposeReference = Settlement.id"
82
```

Settlements		
UNIQUEIDENTIFIER	id	PK
VARCHAR	accountId	
NVARCHAR	status	
NVARCHAR	type	
DECIMAL	amount	
DECIMAL	invoiceTotal	
DECIMAL	creditNoteTotal	
DATETIME2	creationDateTime	
DATETIME2	updationDateTime	



SettlementsAllocation		
UNIQUEIDENTIFIER	id	PK
UNIQUEIDENTIFIER	settlementId	FK
NVARCHAR	invoiceId	
DATETIME2	creationDateTime	
DATETIME2	updationDateTime	

Payments		
UNIQUEIDENTIFIER	Id	PK
UNIQUEIDENTIFIER	CorrelationId	
UNIQUEIDENTIFIER	SystemPaymentMethodId	FK
NVARCHAR	DeviceId	
NVARCHAR	SessionId	
UNIQUEIDENTIFIER	UserId	
UNIQUEIDENTIFIER	SystemId	
VARCHAR	AccountId	
NVARCHAR	PaymentPurpose	
UNIQUEIDENTIFIER	PaymentPurposeReference	
CHAR	CurrencyCode	
DECIMAL	TotalAmount	
NVARCHAR	ResultPageUrl	
NVARCHAR	PaymentStatus	
DATETIME	CreationDatetime	
DATETIME	UpdationDatetime	

invoiceId / logical relationship

14. Step 1 – User Starts Payment

The user selects:

```
1 | Make a payment
```

The system displays two options:

```
1 |
2 | Make a payment
3 |
4 | Select invoices to pay
5 |
6 | Pay custom amount
7 |
```

The user chooses one of these paths.

15. Step 2 – Invoice Payment Path

If the user chooses:

```
1 | Select invoices to pay
```

the system retrieves available invoices.

Only:

```
1 | documentType = 'S'
```

should be returned.

Conceptually:

```
1 | SELECT * FROM dbo.Invoices_Headers WHERE customerAccount = @AccountId AND documentType = 'S';
```

16. Step 3 – User Selects Invoices

The user selects one or more invoices.

For example:

Invoice	Amount
4005/00265187	£1,245.99
4005/00265734	£1,876.50
4005/00265892	£3,499.00

The system now calculates the amount for each invoice.

17. Step 4 – Calculate Invoice Amount

For each selected invoice:

```
1 | Invoices_Headers
```

```

2      |
3      v
4 Orders_Lines
5      |
6      v
7 Orders_Line_Pricings

```

Calculate:

```

1 Invoice Amount = SUM(Quantity × Price) + VAT

```

The calculated value is stored in:

```

1 SettlementInvoiceAllocation.InvoiceAmount

```

and normally:

```

1 AllocatedAmount = InvoiceAmount

```

for full invoice payment.

18. Step 5 – Create Settlement

After invoice selection, create the settlement:

```

1 Settlement

```

Example:

```

1 Id           = S123
2 AccountId    = ACC001
3 PaymentType  = INVOICE
4 Status       = DRAFT
5 CurrencyCode = GBP

```

At this point:

```

1 InvoiceTotal      = 0
2 CreditNoteTotal  = 0
3 PayableAmount    = 0

```

The allocations are then added.

19. Step 6 – Save Invoice Allocations

For three invoices:

```

1 SettlementsAllocation
2
3 SettlementId  InvoiceId  InvoiceAmount  AllocatedAmount
4 -----
5 S123          INV001    1,245.99      1,245.99
6 S123          INV002    1,876.50      1,876.50
7 S123          INV003    3,499.00      3,499.00

```

Then:

```

1 InvoiceTotal = 1,245.99 + 1,876.50 + 3,499.00 = £6,621.49

```

20. Step 7 – Display Credit Notes

Now the system searches for available credit notes.

The same table is used:

```
1 | Invoices_Headers
```

but this time:

```
1 | documentType = 'C'
```

Example:

1	Credit Note	Amount
2		
3	CN001	£956.60
4	CN002	£1,000.00

21. Step 8 – User Selects Credit Notes

The user selects the credit notes they want to apply.

For example:

1	CN001	£956.60
2	CN002	£1,000.00

The system creates:

```
1 | SettlementCreditNoteAllocation
```

22. Step 9 – Calculate Credit Note Total

```
1 | CreditNoteTotal = SUM(AppliedAmount)
```

Example:

```
1 | £956.60 + £1,000.00 = £1,956.60
```

Settlement now contains:

1	InvoiceTotal	= £6,621.49
2	CreditNoteTotal	= £1,956.60

23. Step 10 – Calculate Payable Amount

```
1 | PayableAmount = InvoiceTotal - CreditNoteTotal
```

Therefore:

```
1 | £6,621.49 - £1,956.60 = £4,664.89
```

The UI displays:

1	Subtotal	£6,621.49
2	Credit notes	-£1,956.60

```
3 -----
4 Total                £4,664.89
```

19. Step 11 – Select Payment Method

```
1 Visa
```

The selected payment method is passed to the existing payment service.

24. Step 12 – Create Payment

The Settlement Service creates a record in the existing:

```
1 Payments
```

Important fields:

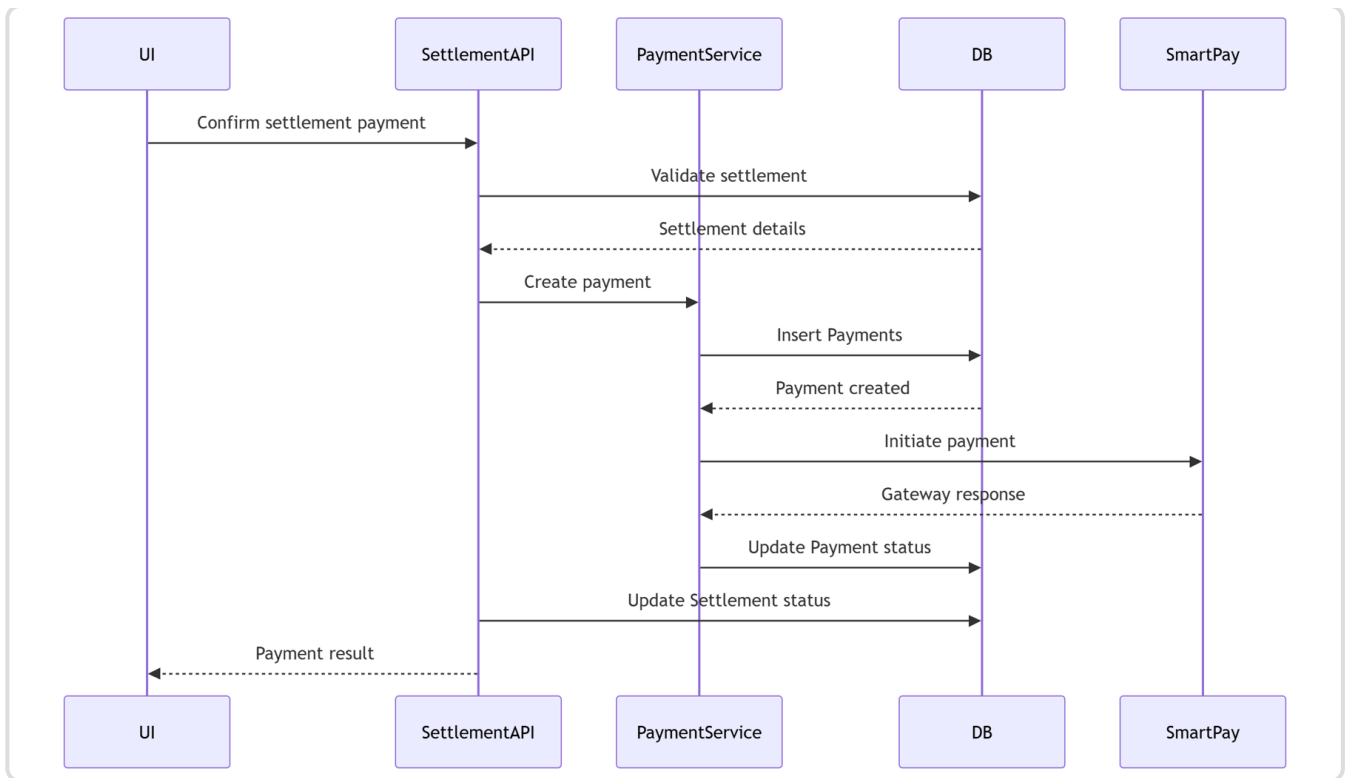
```
1 PaymentPurpose = SETTLEMENT
2 PaymentPurposeReference = Settlement.Id
3 TotalAmount = Settlement.PayableAmount
```

For example:

```
1 Payments
2
3 PaymentPurpose      SETTLEMENT
4 PaymentPurposeReference S123
5 TotalAmount         £4,664.89
6 PaymentStatus       IN_PROGRESS
```

Flow:

```
1 sequenceDiagram
2
3     participant UI
4     participant SettlementAPI
5     participant PaymentService
6     participant DB
7     participant SmartPay
8
9     UI->>SettlementAPI: Confirm settlement payment
10
11    SettlementAPI->>DB: Validate settlement
12    DB-->>SettlementAPI: Settlement details
13
14    SettlementAPI->>PaymentService: Create payment
15
16    PaymentService->>DB: Insert Payments
17    DB-->>PaymentService: Payment created
18
19    PaymentService->>SmartPay: Initiate payment
20    SmartPay-->>PaymentService: Gateway response
21
22    PaymentService->>DB: Update Payment status
23
24    SettlementAPI->>DB: Update Settlement status
25
26    SettlementAPI-->>UI: Payment result
```



25. Step 14 – Successful Payment

If SmartPay succeeds:

```
1 Payments.PaymentStatus = SUCCESSFUL
```

and:

```
1 Settlement.Status = SETTLED
```

Final state:

```

1 Settlement
2 -----
3 Status = SETTLED
4 PayableAmount = £4,664.89
5
6 Payments
7 -----
8 PaymentStatus = SUCCESSFUL
9 TotalAmount = £4,664.89

```

26. Step 15 – Failed Payment

If SmartPay fails:

```
1 Payments.PaymentStatus = FAILED
```

and the settlement should not be marked as paid.

For example:

```

1 Settlement
2 Status = DRAFT

```

```
3
4 Payments
5 PaymentStatus = FAILED
```

The settlement can then either be retried according to the agreed business rules or cancelled.

27. Custom Amount Flow

Now separately document the second flow.

The user selects:

```
1 Pay custom amount
```

The user enters:

```
1 Amount:
2 £999.00
```

and:

```
1 Payment reference:
2 Invoice 4005/00265954
```

28. Custom Settlement

Create:

```
1 Settlements
```

with:

```
1 PaymentType = CUSTOM
2 Amount = £999.00
3 CustomPaymentReference = Invoice 4005/00265954
```

There is **no requirement to create an invoice allocation** for this flow.

29. Custom Payment

The payment is created exactly the same way:

```
1 Payments
2
3 PaymentPurpose = SETTLEMENT
4 PaymentPurposeReference = Settlement.Id
5 TotalAmount= £999.00
```

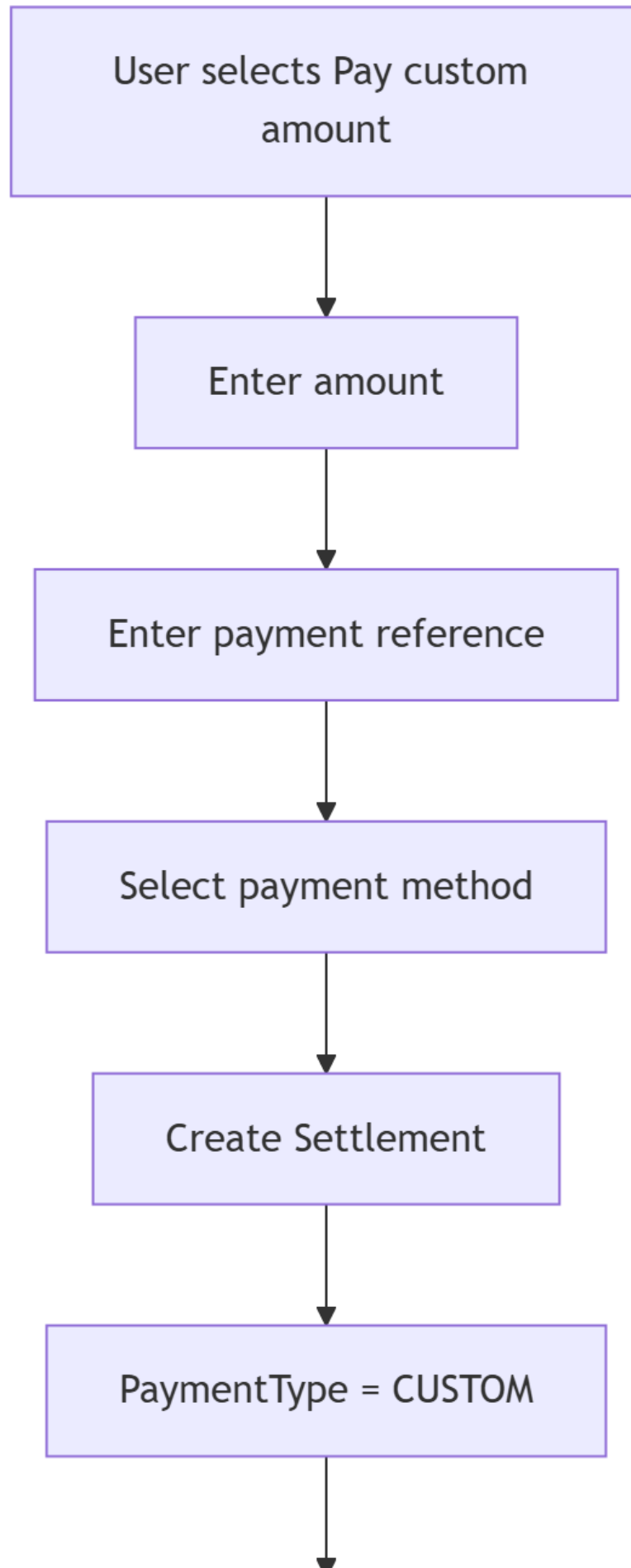
Then:

```
1 Payment
2 |
3 v
4 SmartPay
```

30. Custom Payment Flow Diagram

```
1 flowchart TD
```

```
2
3 A[User selects Pay custom amount]
4 B[Enter amount]
5 C[Enter payment reference]
6 D[Select payment method]
7 E[Create Settlement]
8 F[PaymentType = CUSTOM]
9 G[CustomAmount = entered amount]
10 H[PayableAmount = CustomAmount]
11 I[Create Payment]
12 J[PaymentPurpose = SETTLEMENT]
13 K[PaymentPurposeReference = Settlement.Id]
14 L[Send to SmartPay]
15 M{Payment result}
16
17 A --> B
18 B --> C
19 C --> D
20 D --> E
21 E --> F
22 F --> G
23 G --> H
24 H --> I
25 I --> J
26 J --> K
27 K --> L
28 L --> M
29
30 M -->|Success| N[Payment SUCCESSFUL]
31 N --> O[Settlement PAID]
32
33 M -->|Failure| P[Payment FAILED]
```



CustomAmount = entered
amount



PayableAmount =
CustomAmount



Create Payment



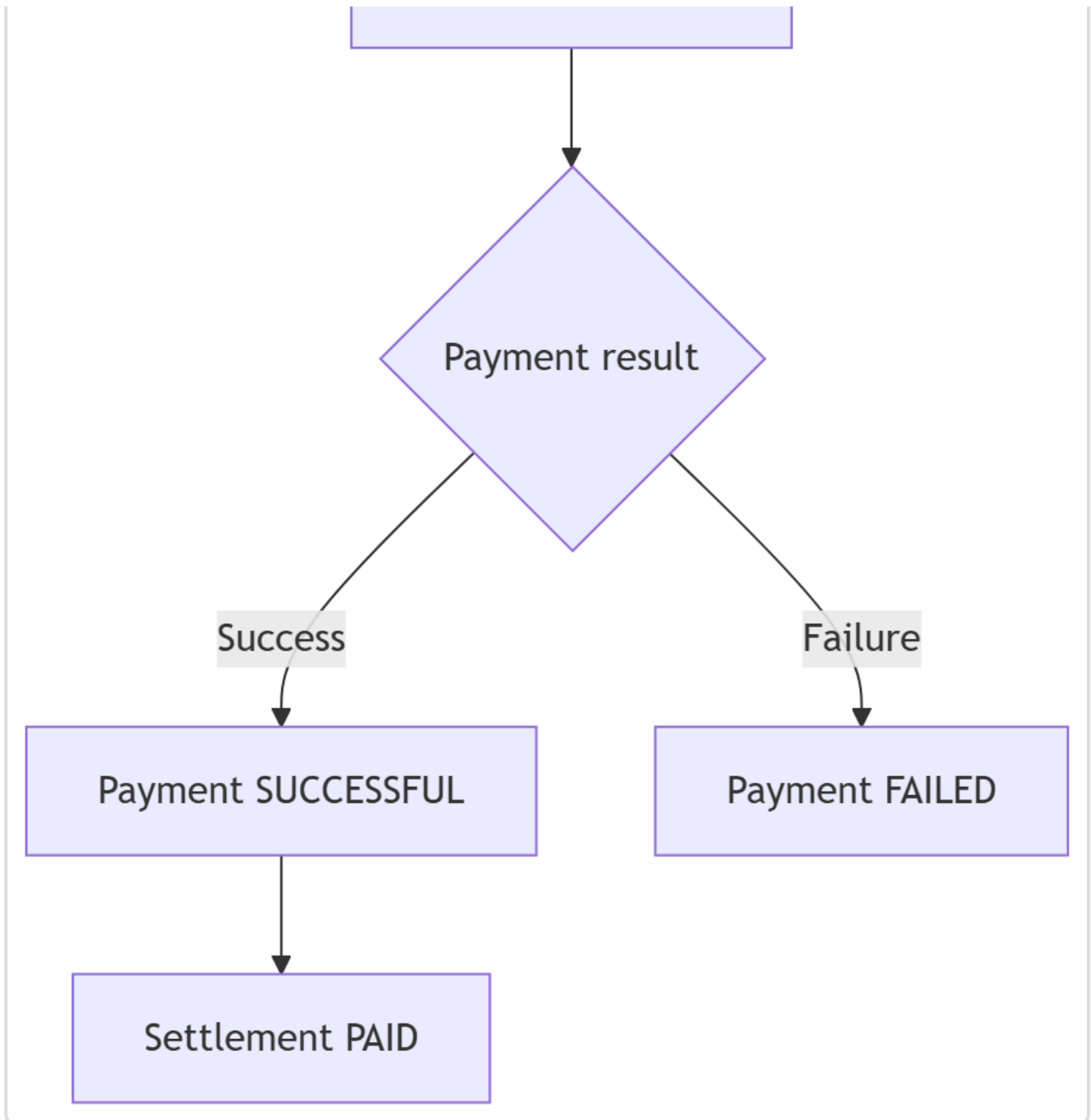
PaymentPurpose =
SETTLEMENT



PaymentPurposeReference
= Settlement.Id



Send to SmartPay



31. Settlement Status

Settlement status is separate from Payment status.

Recommended:

```
1 DRAFT
2 |
3 v
4 LOCKED
5 |
6 +----> SETTLED
7 |
8 +----> PAYMENT_FAILED
```

Potentially:

```

1 DRAFT
2 |
3 +----> CANCELLED

```

32. Payment Status

Payment status is independent:

```

1 IN_PROGRESS
2 |
3 +----> SUCCESSFUL
4 |
5 +----> FAILED

```

This is important because one settlement could potentially have multiple payment attempts.

Example:

```

1 Settlement S123
2 |
3 +---- Payment P001 -> FAILED
4 |
5 +---- Payment P002 -> SUCCESSFUL

```

Therefore:

```

1 Settlement

```

represents the business transaction, while:

```

1 Payments

```

represents payment attempts.

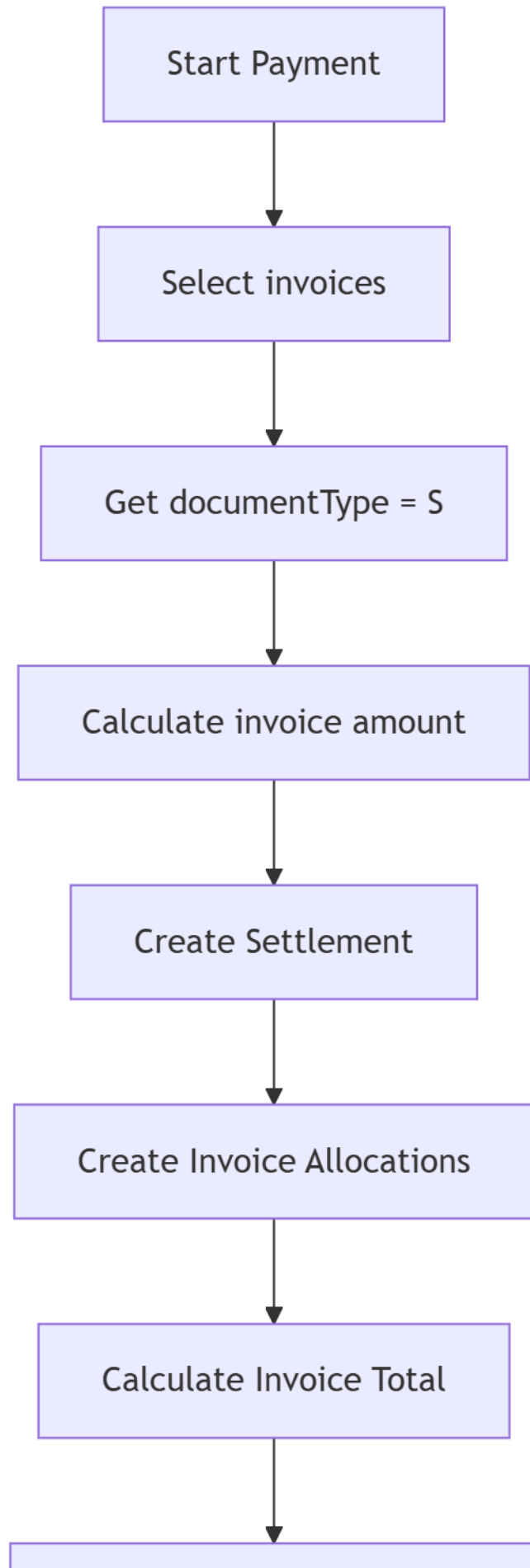
33. Complete Invoice Flow

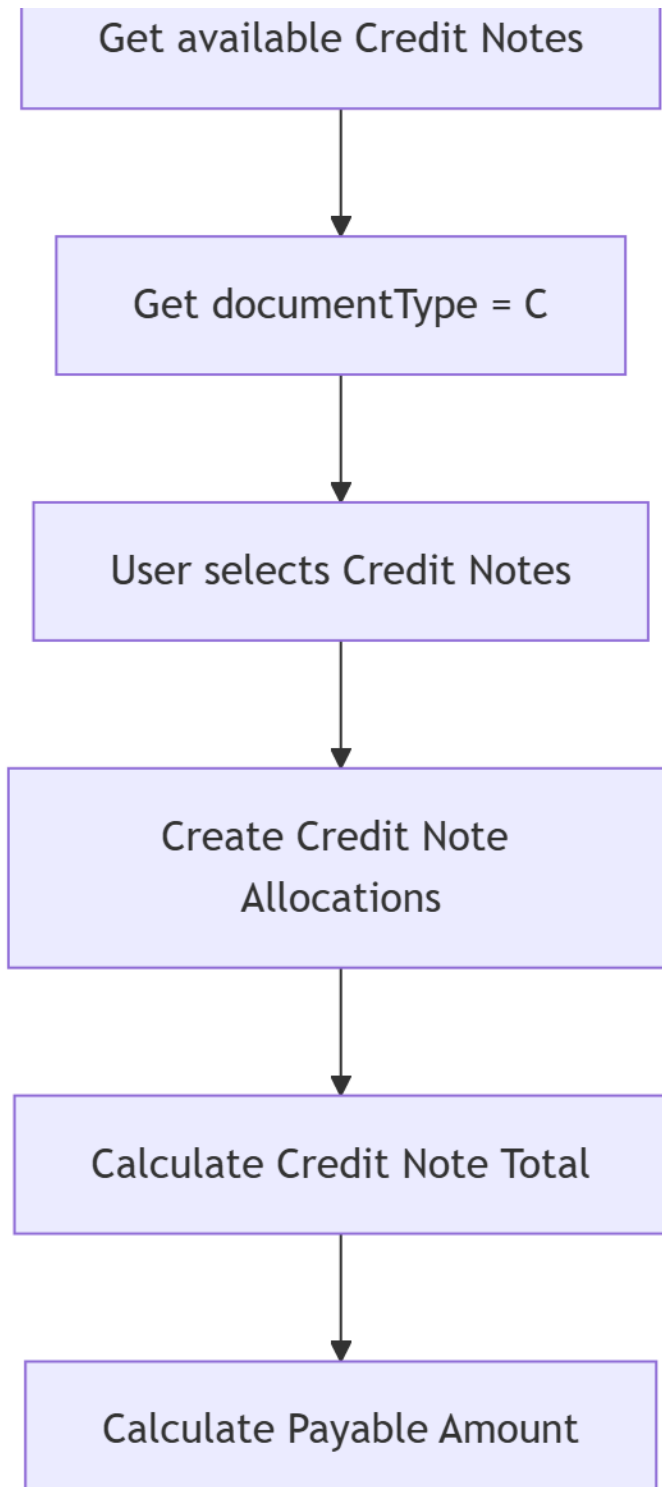
```

1 flowchart TD
2
3   A[Start Payment]
4
5   A --> B[Select invoices]
6
7   B --> C[Get documentType = S]
8
9   C --> D[Calculate invoice amount]
10
11  D --> E[Create Settlement]
12
13  E --> F[Create Invoice Allocations]
14
15  F --> G[Calculate Invoice Total]
16
17  G --> H[Get available Credit Notes]
18
19  H --> I[Get documentType = C]
20
21  I --> J[User selects Credit Notes]
22
23  J --> K[Create Credit Note Allocations]
24
25  K --> L[Calculate Credit Note Total]

```

```
26
27   L --> M[Calculate Payable Amount]
28
29   M --> N[User selects Payment Method]
30
31   N --> O[Create Payment]
32
33   O --> P[SmartPay]
34
35   P --> Q{Result}
36
37   Q -->|Success| R[Payment SUCCESSFUL]
38   R --> S[Settlement PAID]
39
40   Q -->|Failure| T[Payment FAILED]
41   T --> U[Settlement PAYMENT_FAILED]
```





34. Complete Database Flow

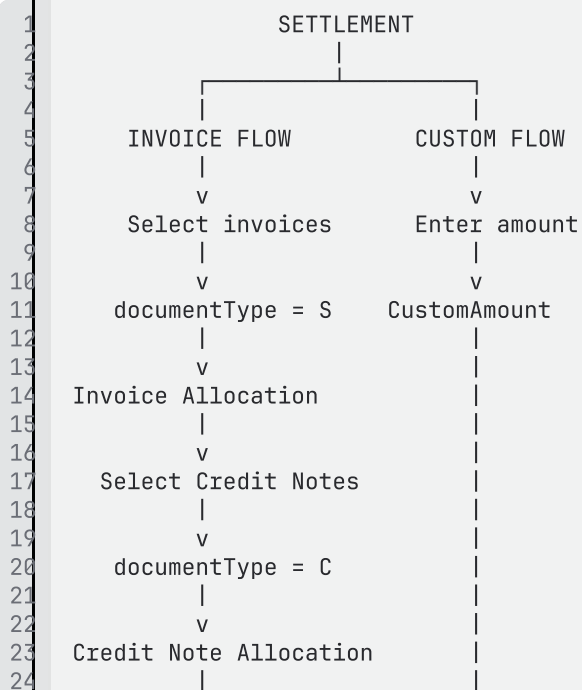
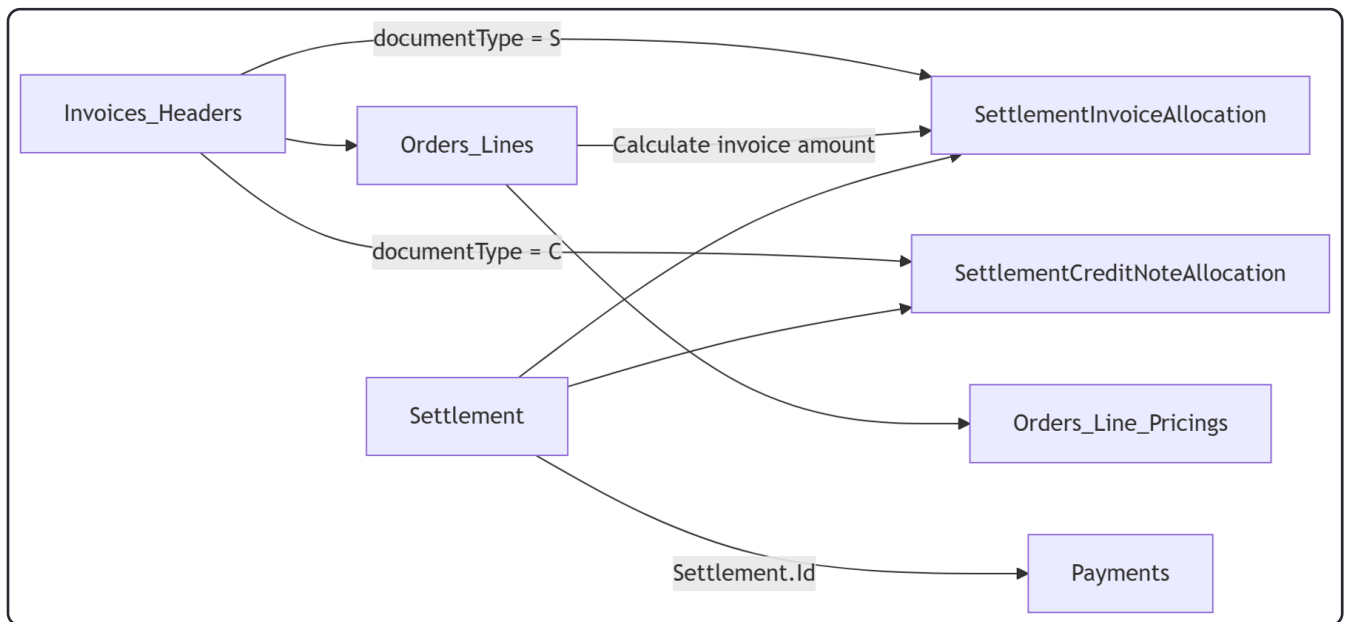
The data movement is:

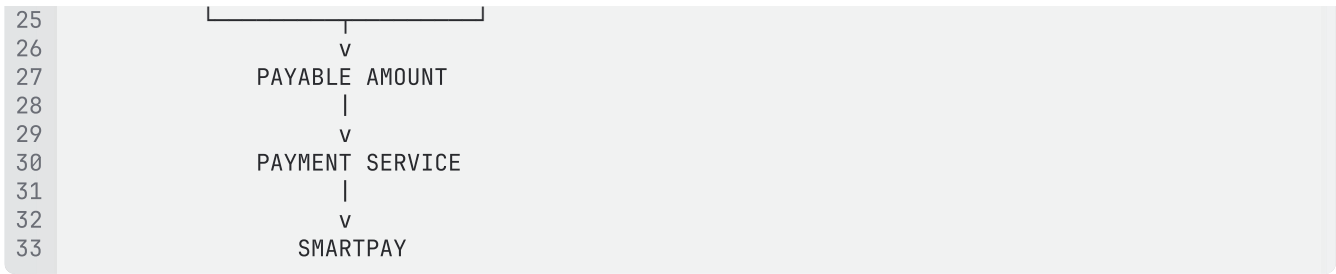
```
1 flowchart LR
2
3   IH[Invoices_Headers]
4
5   OL[Orders_Lines]
6
7   OLP[Orders_Line_Pricings]
8
9   S[Settlement]
10
```

```

11 SIA[SettlementInvoiceAllocation]
12
13 SCA[SettlementCreditNoteAllocation]
14
15 P[Payments]
16
17 IH -->|documentType = S| SIA
18 IH -->|documentType = C| SCA
19
20 IH --> OL
21 OL --> OLP
22
23 OL -->|Calculate invoice amount| SIA
24
25 S --> SIA
26 S --> SCA
27
28 S -->|Settlement.Id| P

```





Swagger OPENAPI Specs:

```
swagger: "2.0"
info:
  title: Settlement API
  version: 1.0.0
  description: Settlement API
  contact:
    name: Settlement API
    email: settlement-api@...er.yaml
    url: https://...er.yaml
  license:
    name: Settlement API
    url: https://...er.yaml
paths:
  /settlement-api-...er.yaml:
    get:
      summary: Get Settlement API
      description: Get Settlement API
      responses:
        200:
          description: Settlement API
          content:
            application/json:
              schema:
                type: object
                properties:
                  settlement-api-...er.yaml:
                    type: string
                    example: settlement-api-...er.yaml
```